

平时作业二

2312966 林晖鹏 计算机学院

一、为什么 `if (p1 > p2)` 可能会生成 `jbe` 这样的指令？

这个问题第一眼看上去确实有点反直觉。源代码里写的是：

```
if (p1 > p2)
```

直觉上觉得编译器应该生成“大于就跳转”的指令，比如 `ja` 或者 `jg`，但是实际汇编里却出现了 `jbe`。其实这完全是合理的，因为编译器不一定非要在条件成立时才跳转，有时候（应该是看哪个更方便，应该是一个小优化加速）会选择在条件不成立时直接跳到 `else` 分支。

关注 PPT 里的这段汇编：

```
movl 8(%ebp), %eax
movl 12(%ebp), %edx
cmpl %edx, %eax
jbe .L1
```

这里一定要注意 AT&T 汇编的语法特点。`cmpl src, dst` 实际做的是：**`dst - src`**

所以：

```
cmpl %edx, %eax
```

比较的是：**`%eax - %edx`**

也就是在算 `p1 - p2`。

然后看 `jbe`，它的意思就是“jump if below or equal”，无符号比较下“小于等于就跳转”。所以：

```
jbe .L1
```

它表达的逻辑是：如果 `p1 <= p2`，就跳到 `.L1`

这样一来，源代码里的 `if (p1 > p2)` 并没有被翻译错，而是编译器用了“反条件跳转”的技巧。原本的代码逻辑完全可以等价地写成：

```
if (!(p1 > p2)) goto else_branch;
// then 分支
```

而 `!(p1 > p2)` 正好就是 `p1 <= p2`，所以当 `p1 <= p2` 时跳到 `else` 分支；没有跳转的话，说明 `p1 > p2` 成立，就顺序执行 `then` 分支。

还有一点需要注意，这里用的是 `jbe`，而不是有符号比较的 `jle`。这是因为 `p1` 和 `p2` 都是指针，指针比较本质上是比地址值，地址当作无符号数来理解更自然，所以编译器用的是 `ja / jbe` 这组无符号条件跳转，而不是 `jb / jle` 那组有符号的。

二、`/lib/ld-linux.so.2` 里的 `.2` 到底是什么意思？

`/lib/ld-linux.so.2` 是 Linux 下面的动态链接器，也就是动态装载器。当一个动态链接的 ELF 程序启动时，并不是直接就进到 `main()` 函数里的，而是先由这个动态链接器接手，它负责把程序依赖的那些共享库（比如 `libc.so`、`libm.so` 之类）都加载好，再搞定符号解析和重定位，最后才会把控制权交给我们写的程序。

那个文件名里尾巴上的 `.2` 并不是普通文件扩展名，而是共享对象的主版本号，也可以理解成 ABI 的主版本号。它表示这个动态链接器对外提供的是哪一代的接口，也就是说，它和哪些二进制程序是兼容的。

简单来讲，`/lib/ld-linux.so.2` 可以理解为：**32 位 x86 Linux 程序常用的第 2 代 ABI 版本的动态链接器**

这个 `2` 最核心的作用就是为了管理兼容性。只要 ABI 还没变，系统内部怎么修补、怎么小版本更新，都可以继续沿用同一个主版本号；假如将来 ABI 发生了不兼容的大变化，那就得换成新主版本号的动态链接器了。这样就能做到老程序继续找老的动态链接器，新程序用新的版本，两不耽误。

另外多说一句，`/lib/ld-linux.so.2` 是和 32 位 Linux 程序绑在一起的。如果你在 64 位系统上尝试跑一个 32 位程序，结果报错说找不到它，那绝大多数情况是因为系统里还没有装对应的 32 位运行库。对于 64 位 x86-64 程序，常看到的动态链接器路径一般是类似这样的：`/lib64/ld-linux-x86-64.so.2`

再结合我们这门课里的工程来看的话，还要稍微区分一下普通的 Linux 用户态程序和像 `x86-nemu` 这样的目标。`x86-nemu` 虽然也是 32 位 i386 的目标，但它更像是给 NEMU 加载的一个裸机镜像，不一定是那种靠 `/lib/ld-linux.so.2` 启动的普通动态链接程序。所以，这里的 `.2` 表示的就是动态链接器的 ABI 主版本号，并不是在说所有 32 位目标都得依赖这个文件。